



**Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026**

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251233
Nama Lengkap	Mikael Gratianus Satrio Adi Kuncoro
Minggu ke / Materi	08 / Pengolahan String dan Regular Expression

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026**

Pengantar String

String adalah kumpulan karakter (huruf, angka, atau simbol) yang digabung menjadi satu untuk menyimpan teks dalam program. String bisa berupa kata atau kalimat dan disimpan dalam kode ASCII. Tidak semua bahasa pemrograman memiliki tipe data string, misal nya seperti bahasa C. Tipe data ini merupakan jenis tipe data yang bukan tipe data dasar, karena tipe data ini pada dasarnya menyimpan lebih dari satu nilai tunggal sebagai satu kesatuan. Di beberapa bahasa pemrograman, string dianggap sebagai kumpulan karakter (seperti array atau list).

Pengaksesan String dan Manipulasi String

String dapat dibuat dengan sederhana menggunakan variabel:

```
1  namesaya = "Antonius Rachmat C"
2  temansaya1 = "Yuan Lukito"
3  temansaya2 = 'Laurentius Kuncoro'
4  temansaya3 = "Matahari" + 'Bakti'
5  print(temansaya3)
6  print(namesaya[0]) #'A'
7  print(namesaya[9]) #'R'
8  print(temansaya1[1]) #'u'
9  huruf = temansaya2[0]
10 print(huruf) #'L'
```

Gambar 1.1: Contoh *source code* penggunaan string

Jadi, string dibuat dengan deklarasi variabel dan langsung diisi dengan data, string dapat diakses sebagai satu kesatuan dengan menyebut nama variabelnya, atau per huruf dengan menyebutkan indeksnya. Indeks pada string dimulai dari 0 seperti layaknya pada list. Indeks string harus bilangan bulat tidak boleh pecahan.

Di memory komputer, string disimpan dengan urut menggunakan list yang berisi huruf-huruf dengan indeks yang dimulai dari nol.

b	a	n	a	n	a
[0]	[1]	[2]	[3]	[4]	[5]

Gambar 1.2: Bentuk string di dalam memory

Operator dan Metode String

OPERATOR in

Di tipe data string dapat memeriksa apakah suatu kalimat merupakan substring dari suatu kalimat lain menggunakan operator in, yang hasilnya adalah True/False.

```
1  kalimat = "saya mau makan"
2  data = "saya"
3  print(data in kalimat) #True
4  print("mau" in kalimat) #True
5  print("dia" in kalimat) #False
```

Gambar 1.2: Contoh *source code* penggunaan operator in

Selain operator in, pada string juga dapat dilakukan perbandingan (*comparison*), yang hasilnya juga True/False.

```

1  if "saya" > "dia":
2      print("Ya") #Ya
3  else:
4      print("Tidak")
5  if "dua" == "dua":
6      print("Sama") #Sama

```

Gambar 1.3: Contoh *source code* string comparison

FUNGSI len

Dengan menggunakan fungsi `len(<string>)` programmer dapat mengetahui berapa panjang (berapa jumlah karakter) dari sebuah string. Untuk menampilkan huruf terakhir dari sebuah string maka harus menggunakan indeks string yang ke- `len(<string>)`, sebab indeks dimulai dari 0. Berikut contoh *source code* dalam program Python:

```

1  kalimat = "universitas kristen duta wacana yogyakarta"
2  print(len(kalimat)) #output 42
3  terakhir = kalimat[len(kalimat)-1]
4  print(terakhir) #output 'a'
5  #bisa juga menggunakan indeks-1
6  terakhir_versi2 = kalimat[-1]
7  print(terakhir_versi2) #output 'a'
8  #atau menggunakan indeks-2 untuk huruf terakhir kedua
9  terakhir2 = kalimat[-2]
10 print(terakhir2) #output 't'

```

Gambar 1.4: Contoh *source code* penggunaan len

TRAVERSING STRING

Jika ingin menampilkan string dengan cara menampilkan huruf demi huruf adalah dengan menggunakan loop yang dilakukan perhuruf (Dilakukan dengan akses terhadap indeks dan dilakukan tanpa akses terhadap indeks secara otomatis).

<pre> 1 #Dilakukan dengan akses terhadap indeks 2 kalimat = "indonesia jaya" 3 i = 0 4 while i < len(kalimat): 5 print(kalimat[i],end='') 6 i += 1 </pre>	<pre> 1 #Dilakukan tanpa akses terhadap indeks secara otomatis 2 kalimat = "indonesia jaya" 3 for kal in kalimat: 4 print(kal,end='') 5 6 </pre>
--	--

Gambar 1.5: Contoh *source code* traversing string.

STRING SLICE

String slice adalah cara mengambil sebagian (substring) dari string menggunakan indeks dengan format `<string>[awal:akhir]`, di mana indeks dimulai dari 0 dan bagian awal/akhir boleh dikosongkan.

```

1  kalimat = "cerita rakyat"
2  awal = 0
3  akhir = 6
4  print(kalimat[awal:akhir]) #cerita
5  print(kalimat[7:len(kalimat)]) #rakyat
6  print(kalimat[:5]) #cerit
7  print(kalimat[5:]) #a rakyat
8  print(kalimat[:]) #cerita rakyat

```

Gambar 1.6: Contoh *source code* string slice.

- String adalah data bersifat immutable yang arti nya data tersebut tidak bisa diubah saat program berjalan, hanya bisa inisialisasi saja. Contoh:

```

1  kalimat = "satu"
2  kalimat[0] = "batu" #TypeError: 'str' object does not support item assignment
3  print(kalimat)
4
5  #Agar bisa diubah, maka harus disimpan dalam variabel yang berbeda.
6
7  kalimat = "satu"
8  kalimat_baru = kalimat[0] + "alah"
9  print(kalimat_baru) #salah

```

Gambar 1.7: String bersifat immutable

Berikut juga adalah beberapa method String yang sering digunakan, fungsi/method yang mengembalikan string, mengembalikan string baru, tidak mengubah yang aslinya, karena string bersifat immutable:

Nama Method	Kegunaan	Penggunaan
capitalize()	untuk mengubah string menjadi huruf besar	string.capitalize()
count()	menghitung jumlah substring yang muncul dari sebuah string	string.count()
endswith()	mengetahui apakah suatu string diakhiri dengan string yang diinputkan	string.endswith()
startswith()	mengetahui apakah suatu string diawali dengan string yang diinputkan	string.startswith()
find()	mengembalikan indeks pertama string jika ditemukan string yang dicari	string.find()
islower() dan isupper()	mengembalikan True jika string adalah huruf kecil / huruf besar	string.islower() dan string.isupper()
isdigit()	mengembalikan True jika string adalah digit (angka)	string.isdigit()
strip()	menghapus semua whitespace yang ada di depan dan di akhir string	string.strip()
split()	memecah string menjadi token-token berdasarkan pemisah, misalnya berdasarkan spasi	string.split()

Gambar 1.8: Beberapa contoh method string

Operator * dan + pada String

Dalam Python, operator + digunakan untuk menggabungkan string, sedangkan * digunakan untuk mengulang string sesuai jumlah yang ditentukan. Perhatikan kode berikut:

```

1  kata1 = "saya"
2  kata2 = "makan"
3  kata3 = kata1 + " " + kata2
4  print(kata3) #hasil adalah penggabungan: saya makan
5  kata4 = "ulang"
6  print(kata4 * 4) #hasil adalah ulangulangulangulang
7  kata4 = "ulang "
8  print(kata4 * 2) #hasil adalah ulang ulang

```

Gambar 1.9: Contoh penggunaan operator * dan + pada string

Parsing String

Parsing string adalah cara menelusuri string bagian demi bagian untuk mendapatkan/menemukan/mengubah bagian string yang diinginkan. Contoh, "Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka". Dari string diatas, diinginkan untuk mengambil tanggal bulan tahun dari kalimat diatas dan disusun ulang menjadi format 08/17/1945. Ini bisa menggunakan cara parsing string, misalnya:

- Berikut contoh *source code* penyelesaian nya:

Gambar 2.0: Parsing string

Regular expression (regex) adalah teknik untuk mempermudah pengolahan string dengan cara mencari, mengganti, atau menghapus teks berdasarkan pola tertentu. Dibandingkan metode biasa seperti `split()` dan `find()`, regex lebih cepat dan fleksibel, meskipun polanya bisa cukup rumit. Python mendukung regex melalui library `re`, salah satu fungsi dasarnya adalah `search()`.

```
1 import re
2 handle = open('coba-coba.txt')
3 count = 0
4 for line in handle:
5     line=line.rstrip()
6     if re.search('From:', line):
7         count += 1
8     print(line)
9 print("Count: ",count)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

```
PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\str_list.py"
Important email From: mikey_aja@gmail.com
From: egicuek@gmail.com
From: hairgy123@gmail.com
Count: 3
PS D:\New folder - Copy\Coding>
```

Jika diinginkan mencari baris yang diawali dengan pola "From", maka kita harus mengubah parameter fungsi search pada re.search menjadi re.search("^From").

```

1 import re
2 handle = open('coba-coba.txt')
3 count = 0
4 for line in handle:
5     line=line.rstrip()
6     if re.search('^From:', line):
7         count += 1
8         print(line)
9 print("Count: ",count)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

PS D:\New folder - Copy\Coding> python -u "d:\New folder - Copy\Coding\str_list.py"
From: egicuek@gmail.com
From: haingy123@gmail.com
Count: 2
PS D:\New folder - Copy\Coding>

```

Gambar 2.2: Penggunaan regex re.search("^")

Kode di atas dapat digantikan juga dengan fungsi string.startswith("From :").

Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Library Python

Sebelum menggunakan fungsi regex perlu diketahui terlebih dahulu special character, escaped character, himpunan karakter, dan kegunaannya pada pola regex. Berikut contoh-contoh nya:

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{ }	Karakter dengan arti khusus dan escaped character	\{ }d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
\$	Dakhiri dengan	this\$	Diakhiri dengan kata this
*	0 s/d tak terhingga karakter	\{ }d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{ }d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{ }d+	Minimal 1 s/d tak terhingga karakter
{ }	Tepat sebanyak yang ada para { }	\{ }d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(sayalkamu)	saya atau kamu sebagai satu kesatuan
	atau	\{ }d \{ }s	1 digit atau 1 spasi

Gambar 2.3: Tabel special character pada Python

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\\bin" "Rain\\b"
\\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\\d
\\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\\D
\\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\\s
\\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\\S
\\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\\w
\\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\\W
\\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\\AThe"
\\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\\Z"

Gambar 2.4: Tabel escaped character pada Regex

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b,m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari pola 2 huruf: 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari pola 1 huruf a-Z

Gambar 2.5: Himpunan karakter pada Regex

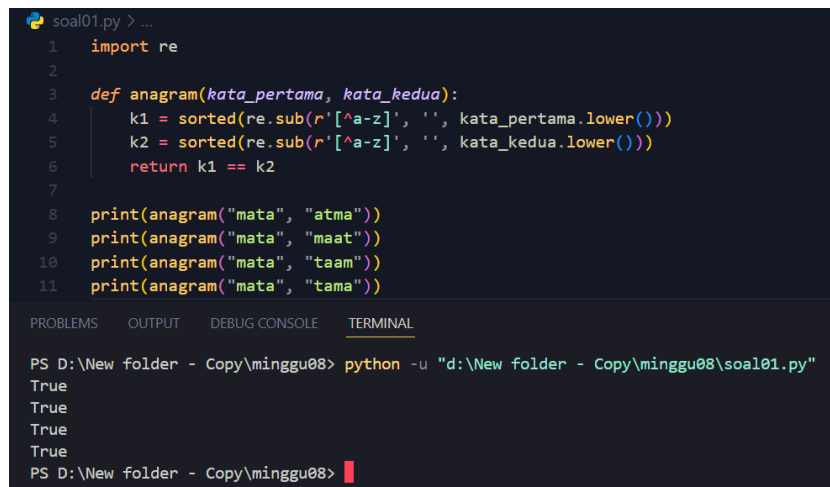
Nama Fungsi	Kegunaan
findall	mengembalikan semua string yang sesuai pola (matches)
search	mengembalikan string yang sesuai pola (match)
split	memecah string sesuai pola
sub	mengganti string sesuai dengan pola yang cocok

Gambar 2.6: Fungsi Regex pada Python

LINK GITHUB

: <https://github.com/Rio-code-07/71251233-Rio-PrakAlpro.git>

SOAL 01



```
soal01.py > ...
1  import re
2
3  def anagram(kata_pertama, kata_kedua):
4      k1 = sorted(re.sub(r'^a-z', '', kata_pertama.lower()))
5      k2 = sorted(re.sub(r'^a-z', '', kata_kedua.lower()))
6      return k1 == k2
7
8  print(anagram("mata", "atma"))
9  print(anagram("mata", "maat"))
10 print(anagram("mata", "taam"))
11 print(anagram("mata", "tama"))

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

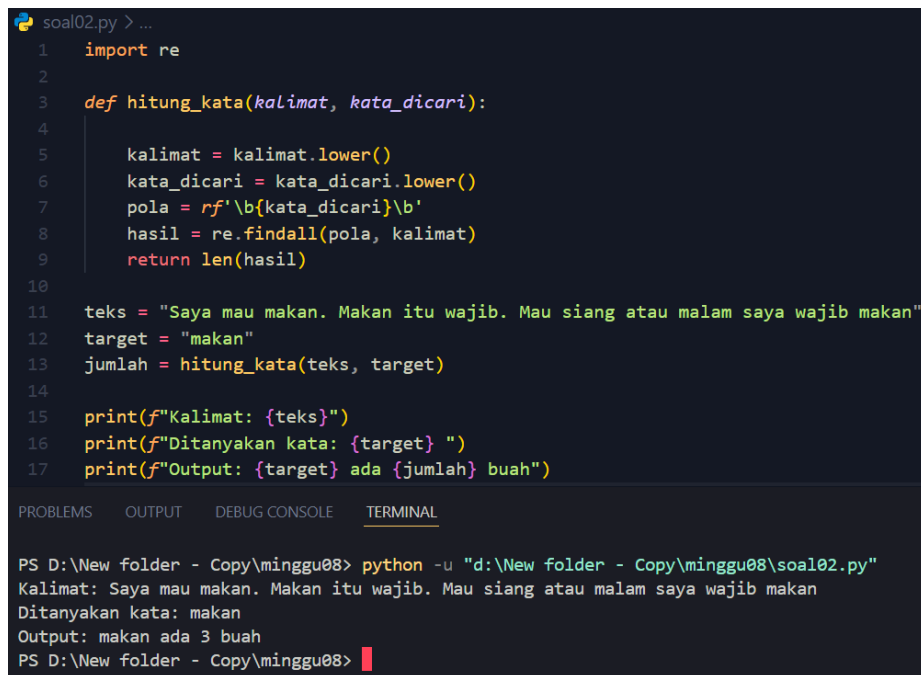
PS D:\New folder - Copy\minggu08> python -u "d:\New folder - Copy\minggu08\soal01.py"
True
True
True
True
PS D:\New folder - Copy\minggu08> █
```

Gambar 2.7: Source code soal 01

Penjelasan:

Ini adalah program untuk mencari anagram. Program dimulai dengan mengimpor library **re** yang digunakan untuk memproses teks dengan pola tertentu, lalu dibuat sebuah fungsi bernama **anagram** yang menerima dua kata sebagai input. Di dalam fungsi tersebut, kedua kata diubah menjadi huruf kecil semua, kemudian semua karakter selain huruf (seperti spasi, angka, dan simbol) dihapus menggunakan **re.sub**, setelah itu huruf-hurufnya diurutkan secara alfabet menggunakan **sorted** dan disimpan ke variabel **k1** dan **k2**. Terakhir, kedua hasil tersebut dibandingkan, jika sama maka fungsi mengembalikan nilai **True** yang berarti kedua kata adalah anagram, jika tidak maka **False**. Pada bagian akhir, fungsi dipanggil dengan beberapa contoh seperti "atma", "maat", "taam", dan "tama", yang semuanya menghasilkan **True** karena hurufnya sama walaupun urutannya berbeda.

SOAL 02



```
soal02.py > ...
1  import re
2
3  def hitung_kata(kalimat, kata_dicari):
4
5      kalimat = kalimat.lower()
6      kata_dicari = kata_dicari.lower()
7      pola = rf'\b{kata_dicari}\b'
8      hasil = re.findall(pola, kalimat)
9      return len(hasil)
10
11 teks = "Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan"
12 target = "makan"
13 jumlah = hitung_kata(teks, target)
14
15 print(f"Kalimat: {teks}")
16 print(f"Ditanyakan kata: {target} ")
17 print(f"Output: {target} ada {jumlah} buah")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS D:\New folder - Copy\minggu08> python -u "d:\New folder - Copy\minggu08\soal02.py"
Kalimat: Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan
Ditanyakan kata: makan
Output: makan ada 3 buah
PS D:\New folder - Copy\minggu08>
```

Gambar 2.8: Source code soal 02

Penjelasan:

Ini adalah program untuk menghitung kata yang telah ditentukan, Program diawali dengan mengimpor library **re** yang digunakan untuk mencari pola dalam teks, lalu dibuat fungsi bernama **hitung_kata** yang menerima dua input yaitu kalimat dan kata yang ingin dicari. Di dalam fungsi, kalimat dan kata_dicari diubah menjadi huruf kecil menggunakan **lower()** agar pencarian tidak terpengaruh huruf besar atau kecil. Setelah itu dibuat pola regex menggunakan **\b (batas kata)** agar hanya mencocokkan kata yang utuh, bukan bagian dari kata lain, kemudian **re.findall** digunakan untuk mencari semua kemunculan kata tersebut dalam kalimat. Hasil berupa list (daftar kata yang ditemukan), namun tidak disimpan lama karena langsung digunakan untuk menghitung jumlahnya dengan **len()**. Fungsi kemudian mengembalikan jumlah kemunculan kata tersebut. Di bagian akhir, sebuah kalimat disimpan ke variabel **teks** dan kata yang ingin dicari disimpan ke variabel **target**, lalu fungsi dipanggil dan hasilnya dicetak, menampilkan kalimat, kata yang dicari, serta jumlah kemunculannya dalam teks.

SOAL 03



```
soal03.py > ...
1  import re
2
3  def hapus_spasi(teks):
4      teks_bersih = re.sub(r'\s+', ' ', teks).strip()
5      return teks_bersih
6
7  teks_kotor = "saya tidak suka memancing ikan "
8  print(f'Output: "{hapus_spasi(teks_kotor)}"')

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

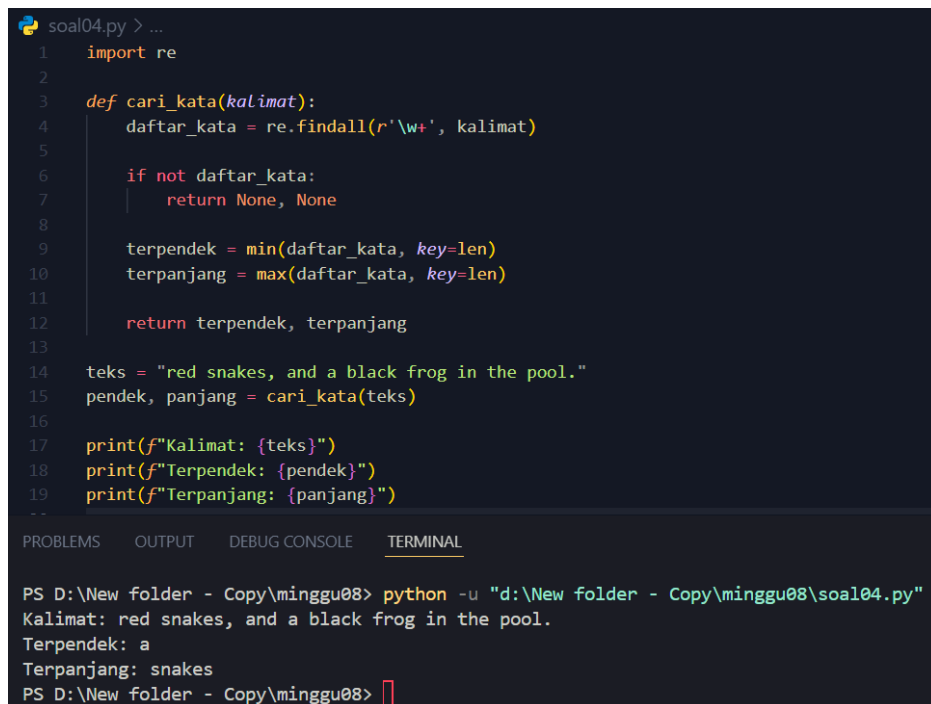
PS D:\New folder - Copy\minggu08> python -u "d:\New folder - Copy\minggu08\soal03.py"
Output: "saya tidak suka memancing ikan"
PS D:\New folder - Copy\minggu08>
```

Gambar 2.9: Source code soal 03

Penjelasan:

Ini adalah program untuk membersihkan spasi berlebih dalam sebuah kalimat. Program diawali dengan mengimpor library **re** yang digunakan untuk memproses teks dengan pola tertentu, lalu dibuat fungsi bernama **hapus_spasi** yang menerima satu parameter yaitu **teks**. Di dalam fungsi, digunakan **re.sub** dengan pola **\s+** untuk mengganti satu atau lebih spasi menjadi hanya satu spasi saja, kemudian dilanjutkan dengan **.strip()** untuk menghapus spasi di bagian awal dan akhir kalimat. Hasilnya disimpan sementara dalam variabel **teks_bersih**, lalu dikembalikan sebagai output fungsi. Di bagian akhir, sebuah kalimat yang masih memiliki spasi berlebih disimpan dalam variabel **teks_kotor**, kemudian fungsi dipanggil langsung di dalam print untuk menampilkan hasil kalimat yang sudah rapi tanpa spasi berlebih.

SOAL 04



```
soal04.py > ...
1  import re
2
3  def cari_kata(kalimat):
4      daftar_kata = re.findall(r'\w+', kalimat)
5
6      if not daftar_kata:
7          return None, None
8
9      terpendek = min(daftar_kata, key=len)
10     terpanjang = max(daftar_kata, key=len)
11
12     return terpendek, terpanjang
13
14     teks = "red snakes, and a black frog in the pool."
15     pendek, panjang = cari_kata(teks)
16
17     print(f"Kalimat: {teks}")
18     print(f"Terpendek: {pendek}")
19     print(f"Terpanjang: {panjang}")
20
21
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
PS D:\New folder - Copy\minggu08> python -u "d:\New folder - Copy\minggu08\soal04.py"
Kalimat: red snakes, and a black frog in the pool.
Terpendek: a
Terpanjang: snakes
PS D:\New folder - Copy\minggu08> 
```

Gambar 3.0: Source code soal 04

Pejelasan:

Ini adalah program untuk mencari kata terpendek dan terpanjang dalam sebuah kalimat. Program diawali dengan mengimpor library **re** yang digunakan untuk mencari pola dalam teks, lalu dibuat fungsi bernama **cari_kata** yang menerima satu parameter yaitu **kalimat**. Di dalam fungsi, digunakan **re.findall** dengan pola **\w+** untuk mengambil semua kata (huruf, angka, dan underscore) dan mengabaikan tanda baca, lalu hasilnya disimpan dalam bentuk list bernama **daftar_kata**. Jika tidak ada kata yang ditemukan, fungsi akan mengembalikan **None, None** agar tidak terjadi error. Selanjutnya, fungsi mencari kata terpendek menggunakan **min** dan kata terpanjang menggunakan **max**, keduanya dengan acuan panjang kata menggunakan **key=len**. Hasilnya kemudian dikembalikan sebagai dua nilai sekaligus. Di bagian akhir, sebuah kalimat disimpan ke variabel **teks**, lalu fungsi dipanggil dan hasilnya ditampung ke variabel **pendek** dan **panjang**, kemudian dicetak untuk menampilkan kalimat asli, kata terpendek, dan kata terpanjang.

SOAL 05

```
soal05.py > ...
1  import re
2  from datetime import datetime
3
4  teks = """Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan
5  nasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Pattimura (TL: 1783-06-08) dan Ki
6  Hajar Dewantara (1889-05-02)."""
7
8  pola = r'(\d{4})-(\d{2})-(\d{2})'
9  daftar_tanggal = re.findall(pola, teks)
10
11  sekarang = datetime.now()
12
13  print("Hasil:")
14  for i in daftar_tanggal:
15      tahun, bulan, hari = i
16
17      objek_tgl = datetime.strptime(f"{tahun}-{bulan}-{hari}", "%Y-%m-%d")
18      selisih = (sekarang - objek_tgl).days
19      tgl_format_baru = objek_tgl.strftime("%d-%m-%Y %H:%M:%S")
20
21      print(f"{tgl_format_baru} selisih {selisih} hari")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

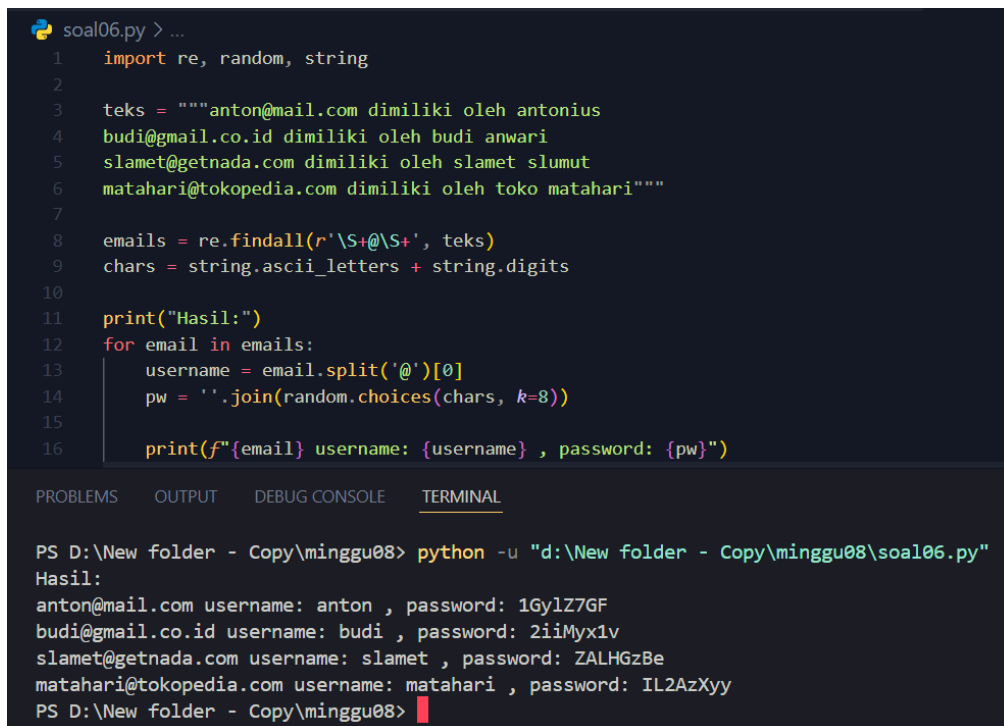
PS D:\New folder - Copy\minggu08> python -u "d:\New folder - Copy\minggu08\soal05.py"
Hasil:
17-08-1945 00:00:00 selisih 29465 hari
11-11-1785 00:00:00 selisih 87817 hari
08-06-1783 00:00:00 selisih 88704 hari
02-05-1889 00:00:00 selisih 50025 hari
PS D:\New folder - Copy\minggu08>
```

Gambar 3.1: Source code soal 05

Penejelasan:

Ini adalah program untuk mencari seluruh teks yang berupa tanggal dengan format yang diubah lalu juga ada perhitungan selisih dengan tanggal sekarang dalam hari. Program dimulai dengan mengimpor **re** untuk pencarian pola dan **datetime** untuk mengolah tanggal, kemudian sebuah teks panjang disimpan dalam variabel **teks** yang berisi beberapa tanggal. Lalu dibuat pola regex **(\d{4})-(\d{2})-(\d{2})** untuk mendeteksi tanggal dengan format tahun-bulan-hari (YYYY-MM-DD), lalu menggunakan **re.findall** semua tanggal yang cocok diambil dan disimpan dalam bentuk list berisi tuple. Selanjutnya, waktu saat program dijalankan diambil dengan **datetime.now()**. Setiap tanggal kemudian diproses dengan perulangan, tuple dipecah menjadi tahun, bulan, dan hari, lalu digabung kembali dan diubah menjadi objek **datetime** menggunakan **strptime()** agar bisa dilakukan perhitungan. Setelah itu dihitung selisih hari antara tanggal tersebut dan sekarang, kemudian tanggal diformat ulang menjadi bentuk yang lebih mudah dibaca menggunakan **strftime()**. Terakhir, hasilnya dicetak berupa tanggal yang sudah diformat beserta jumlah selisih harinya dari sekarang.

SOAL 06



```
soal06.py > ...
1  import re, random, string
2
3  teks = """anton@mail.com dimiliki oleh antonius
4  budi@gmail.co.id dimiliki oleh budi anwari
5  slamet@getnada.com dimiliki oleh slamet slumut
6  matahari@tokopedia.com dimiliki oleh toko matahari"""
7
8  emails = re.findall(r'\S+@\S+', teks)
9  chars = string.ascii_letters + string.digits
10
11 print("Hasil:")
12 for email in emails:
13     username = email.split('@')[0]
14     pw = ''.join(random.choices(chars, k=8))
15
16     print(f"{email} username: {username} , password: {pw}")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS D:\New folder - Copy\minggu08> python -u "d:\New folder - Copy\minggu08\soal06.py"
Hasil:
anton@mail.com username: anton , password: 1GylZ7GF
budi@gmail.co.id username: budi , password: 2iiMyx1v
slamet@getnada.com username: slamet , password: ZALHGzBe
matahari@tokopedia.com username: matahari , password: IL2AzXyy
PS D:\New folder - Copy\minggu08> 
```

Gambar 3.2: Source code soal 06

Penjelasan:

Ini merupakan program untuk mencari seluruh teks yang berupa email dan kemudian ambil semua username dari email tersebut untuk digenerate password random 8 karakter yang terdiri dari angka dan huruf. Program diawali dengan mengimpor **re**, **random**, dan **string**, lalu teks berisi beberapa email disimpan dalam variabel **teks**. Lalu menggunakan **re.findall** dengan pola **\S+@\S+**, semua email dalam teks diambil dan disimpan dalam list. Selanjutnya dibuat kumpulan karakter (**chars**) dari huruf dan angka menggunakan **string.ascii_letters** dan **string.digits** sebagai bahan password. Program kemudian melakukan perulangan untuk setiap email, username diambil dari bagian sebelum tanda **@** menggunakan **split**, lalu password dibuat secara acak sebanyak 8 karakter menggunakan **random.choices** dan digabung menjadi string. Terakhir, hasilnya dicetak berupa email, username, dan password yang dihasilkan.